School of Engineering STI

Model Predictive Control : Mini-project

Rocket controller

By

Antoine Hinary 324960

Nour Lachat 302397

Aubane Lachat 328327

Professor: Colin Jones

Ecole polythechnique fédérale de Lausanne

Date of submission: January 27, 2024

CONTENTS

1 Introduction **2**

2 Deliverable 2.1 **3**

3 Deliverables 3.1 and 3.2 **5**

3.1 Controller Design Procedure : 5

3.2 Tracker Design Procedure: 10

3.3 Tracker Parameter Tuning: 11

4 Deliverables 4.1 and 4.2 **14**

4.1 Simulation with Nonlinear Rocket 14

5 Deliverables 5.1 and 5.2 **16**

5.1 Offset-Free Tracking Controller 16

5.2 System with changing Mass 17

6 Deliverables 6.1 and 6.2 **20**

6.1 Nonlinear MPC 20

6.1.1 Nonlinear Controller vs Linear Controllers 21

6.2 NMPC with Delay Compensation 22

6.2.1 No/Partial/Full delay compensation 23

Appendix **25**

6.3 Runge Kutta 4 Algoritm 25

RK4 25

1. INTRODUCTION

SYSTEM DESCRIPTION

The rocket system under consideration involves a small-scale prototype with high-performance drone racing propellers. The propellers are mounted on a gimbal and can be tilted using servos, allowing for thrust vector control. The 12-state vector x includes angular velocities (ω), Euler angles (ϕ), linear velocities (v), and positions (p). The system is influenced by deflection angles (δ_1, δ_2) and propeller throttle settings ($P_{\text{avg}}, P_{\text{diff}}$).

- Ascend/descend vertically without tipping over : To ensure a straight trajectory of the rocket, input parameters must be set such that δ_1 and δ_2 equal zero, representing constant deviations along the rocket's reference axes x and y to prevent continuous deviations.
- Extending the simulation run-time is crucial for a comprehensive analysis; this is achieved by increasing the simulation time T_f.

States of Interest

- **Angular Velocities (ω):** Represented by $\omega_x, \omega_y, \omega_z$, these describe the rotational motion of the rocket.
- **Euler Angles (ϕ):** Represented by α, β, γ, these define the rocket's attitude with respect to the world frame.
- **Linear Velocities (v):** Represented by v_x, v_y, v_z, these describe the rocket's linear motion.
- **Positions (p):** Represented by x, y, z, these specify the rocket's spatial coordinates.

2. DELIVERABLE 2.1

ANALYSIS OF LINEARIZED SYSTEM

The examination of the linearized system of our rocket model has shown that we can divide our model into four distinct subsystems, each linked to a specific control and thus independent of the other systems. Indeed, intuitively, a rotation around the x-axis doesn't influence whether there's a rotation around the y-axis. These four systems (x-system, y-system, z-system, roll-system) stem from a direct understanding of the mechanical interactions within the system, along with an analysis of the system matrix:

X-system : Thrust Vector Angle δ_2 to Position x

When considering the thrust vector angle δ_2 influencing lateral motion along the x-axis, it becomes evident that this specific control input primarily affects the aircraft's lateral dynamics in that direction. The angular velocity about the y-axis (ω_y), side-slip angle (β), linear velocity in the x-direction (v_x), and position x are logically intertwined within this lateral plane. Breaking down the system here allows for a targeted analysis, honing in on the nuances of lateral motion in response to δ_2. In addition further testing of the simulation given at hand revealed that any modification in the δ_2 within the input vector causes a chain reaction shown within the modification of the x, ω_y and the speed v_x .

Y-system : Thrust Vector Angle δ_1 to Position y

Similarly, the subsystem associated with δ_1 focuses on lateral motion along the y-axis. By isolating this control input, comprising angular velocity about the x-axis (ω_x), angle of attack (α), linear velocity in the y-direction (v_y), and position y, we recognize the independent nature of lateral dynamics in the y-direction. This separation facilitates a specialized examination, capturing the unique influence of δ_1 on lateral motion. We

could also verify the latter discovery by varying the values of the input vector all while maintaining the δ_2 and the P_{diff} null.

Z-system : Average Throttle P_{avg} to Height z

The vertical dynamics controlled by the average throttle, P_{avg}, naturally stand isolated due to the independent nature of vertical motion. States such as vertical velocity (v_z) and height (z) constitute a distinct subsystem. This choice allows for a focused analysis, recognizing the specialized impact of P_{avg} on vertical dynamics. This observation was the first to note while testing the simulation where all inputs were set to 0. Upon introducing throttle power, we observed its direct influence on the altitude. Additionally, augmenting a positive throttle power to a rocket primarily impacts its vertical altitude and speed, given the orientation of the propellers facing the z axis.

Roll-system : Differential Throttle P_{diff} to Roll Angle γ

Delving into the subsystem influenced by differential throttle P_{diff}, which pertains to rotational motion about the longitudinal axis, unveils the intuitive separation of rotational dynamics. The states, including angular velocity about the z-axis (ω_z) and roll angle γ), distinctly capture the influence of P_{diff} on rotational motion. This targeted analysis facilitates a deeper understanding of how differential throttle contributes to roll dynamics. If we look at the system from a pure mechanical point of view we can see that if both throttle powers have a different meaning that both propellers deliver different powers the rocket should normally start rotating around the z axis which means a rotation around ω_z.

3. DELIVERABLES 3.1 AND 3.2

3.1. CONTROLLER DESIGN PROCEDURE :

The design procedure for ensuring recursive constraint satisfaction for our rocket simulation involves a systematic approach to formulate and implement control strategies. The primary goal is to guarantee that the system's constraints are satisfied at each time step, taking into account the dynamic nature of the rocket's individual model behavior. The following steps outline the design procedure:

1. Formulation of Control Objectives and Constraint Matrices:
   Define clear control objectives that align with the desired behavior of the rocket, specifying relevant state variables and control inputs. Simultaneously, formulate constraint matrices (M and F) and their corresponding bounds (m and f) based on the physical limitations of the rocket.

2. Horizon and Cost Matrices:
   Set up horizon and cost matrices (Q and R) that reflect the weighting of state and control variables in the cost function. These matrices play a crucial role in shaping the controller's response, influencing both tracking accuracy and control effort. It is with tuning these matrixes that we can obtain our desired responses for the system.

3. LQR Controller Design:
   Utilize the Linear Quadratic Regulator (LQR) design to compute the state feedback gains (K) and the final cost $(Q_f)$for each subsystem. The LQR controller provides an optimal control strategy that balances state tracking and control effort.

4. Maximal Invariant Set Computation:
   Next, the computation of the maximal invariant set is used aiming to find the

states that permit the system to continuously satisfy requirements across time. By iteratively refining the set until convergence, this stage assures robustness in constraint satisfaction and lays a solid foundation for the system's behaviour.

5. Formulation of Optimization Problem:
   Finally, we set up the optimization problem using YALMIP, defining the objective function and constraints based on the chosen control objectives and constraints. The computed LQR gains and the maximal invariant set are incorporated to enforce recursive constraint satisfaction. We configure the solver settings using the 'gurobi' solver to get our desired control output.

Parameter Tuning :

The tuning parameters, which include Q, R, H, and terminal components, are vital in influencing the controller's behaviour. The reasoning behind their pick is described below:

1. Q and R Matrices:
   - **Q Matrix:** Represents the weighting of state variables in the cost function. Higher values prioritize the importance of specific state variables.
   - **R Matrix:** Represents the weighting of control variables in the cost function. Higher values prioritize the significance of certain control variables.

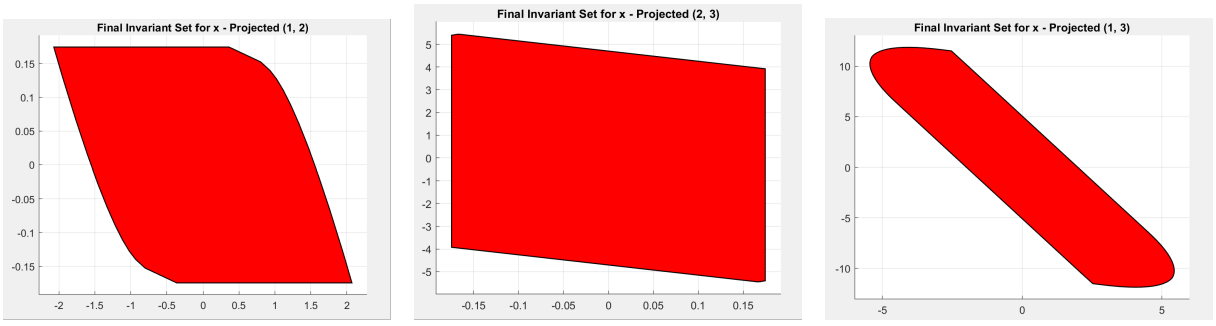
   Tuning Process: The Q and R matrices underwent iterative adjustments to observe their impact on the system's response. Emphasis was placed on giving more weight to constraints for enhanced robustness. We have also noted that the modification made to the R matrix governed the control input allowed, which resulted in modulating the overshoot/the peak of input given to the system. In contrast the Q matrix was mainly used to regulate the importance of the state variable's constraints. For instance we cannot allow that the altitude of the rocket decreases below 0 ($z < 0$) however we can allow a rotation ω_z of around 60 $degrees/s$

2. H :
   - **Prediction Horizon (H):** Determines the number of future time steps considered

in the optimization. We set $H = 10$ for a balance between computational efficiency and predictive capability.

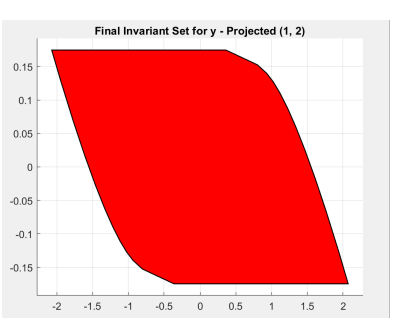
3. Terminal components : The terminal invariant set is computed in two different ways for the systems x, y, and for the systems z and roll. Indeed, as we have constraints on the states and on the input in the x and y system, we can compute it using the following maximal invariant set algorithm:

   - Given the state and input constraints represented by matrices F and M, along with the Linear Quadratic Regulator (LQR) controller gain K, we initialized a polytope Xf with the following constraints $[F; M \cdot K]$ along with $[f; m]$.
   - We then compute the closed-loop system matrix $A_{\text{cl}} = \text{sys_d.A} + \text{sys_d.B} \cdot K$.
   - Iteratively we update Xf until needed convergence is achieved:
     - Record the current polytope as $prevXf = Xf$.
     - Obtain the vertices $[T, t]$ of Xf.
     - Update Xf by intersecting it with the pre-image $prevXf$ under the closed-loop system A_{cl}.
     - Check for convergence by verifying equality between $prevXf$ and the updated Xf.
   - Extract the final constraints $[Ff, ff]$ defining the maximal invariant set Xf.

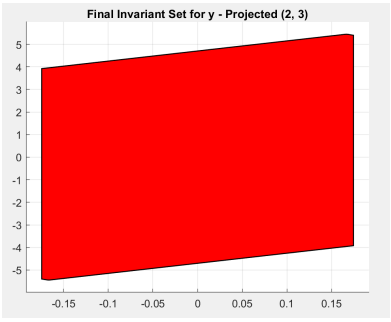


(a) Invariant set for x_{12} **(b)** Invariant set for x_{23} **(c)** Invariant set for x_{13}

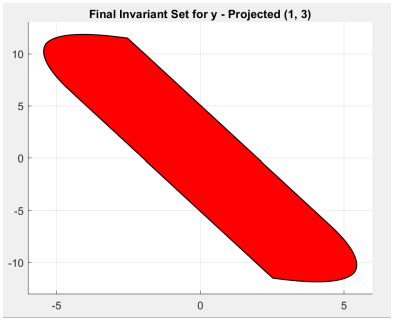
Figure 3.1: Terminal invariant sets for x for 3.1



(a) Invariant set for y_{12}



(b) Invariant set for y_{23}



(c) Invariant set for y_{13}

Figure 3.2: Terminal invariant sets for y for 3.1

As the z and roll systems don't have any state constraints, the following method using the MTP toolbox was employed.

- An LTISystem object is created using this toolbox with the matrices 'A' and 'B' of the system.
- Maximum and minimum values for states ('x' in the code) are set to positive and negative infinity, imposing no state constraints.
- The minimum and maximum value of the input constraints are defined.
- Quadratic penalty functions for states (Q) and inputs (R) are defined using the QuadFunction class from the MPT toolbox.
- The terminal set is computed using the Linear Quadratic Regulator (LQR) approach from the toolbox: `Xf_mpt = sys.LQRSet`.
- Finally the terminal penalty, representing the cost associated with reaching the terminal set, is calculated using the command: `Qf_mpt = sys.LQRPenalty`.

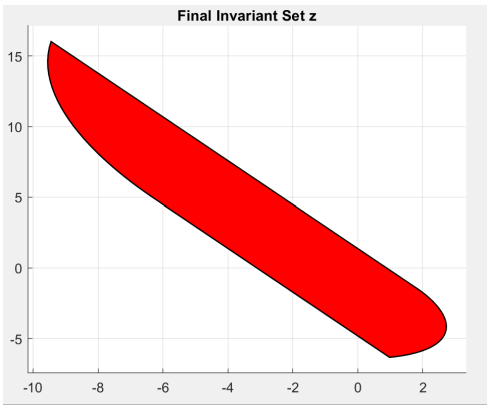


Figure 3.3: Final invariant set for z

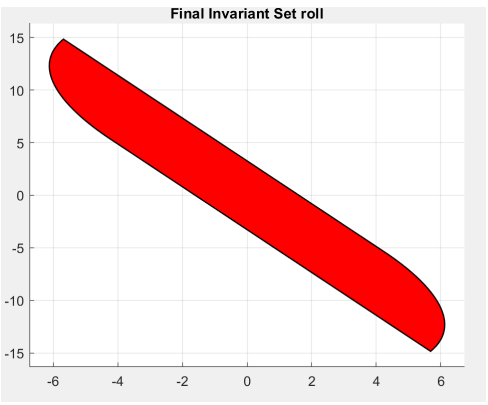
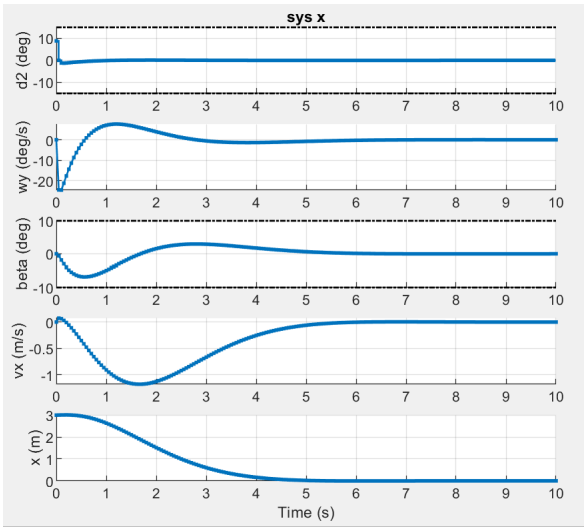
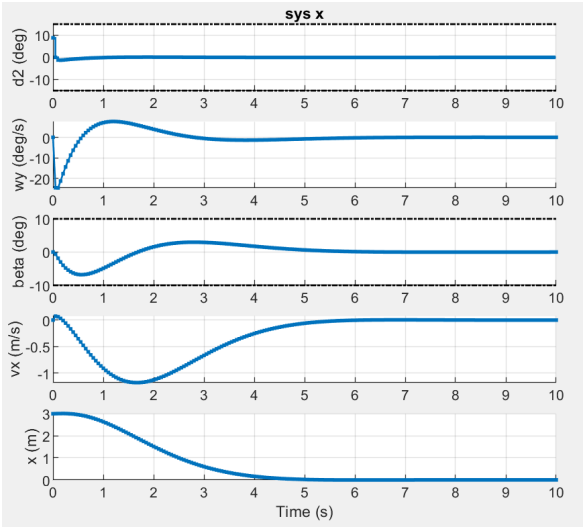


Figure 3.4: Final invariant set for roll

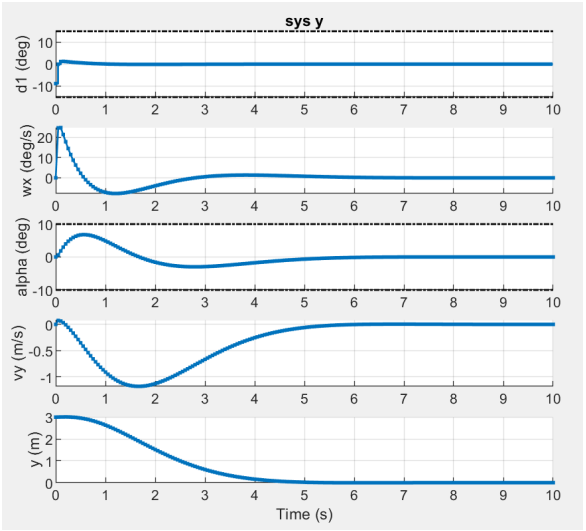
open-loop and closed-loop plot



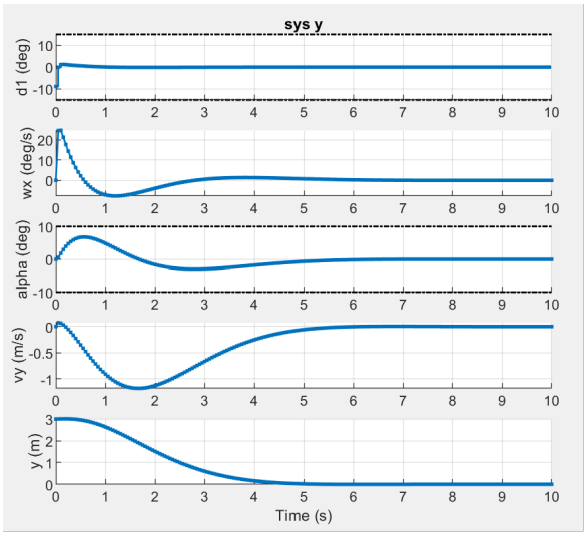
(a) open-loop for x



(b) closed-loop for x

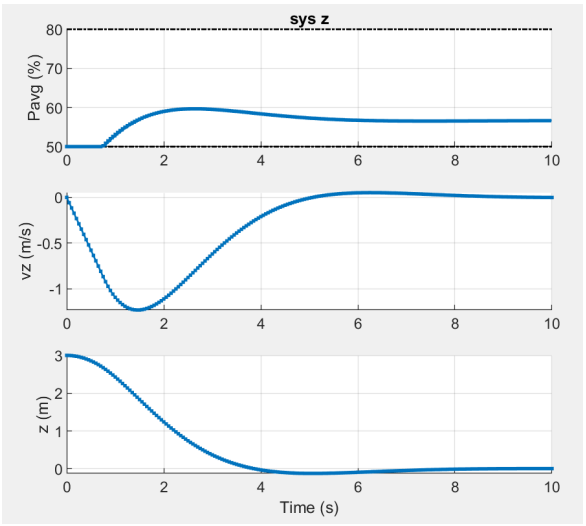


(c) open-loop for y

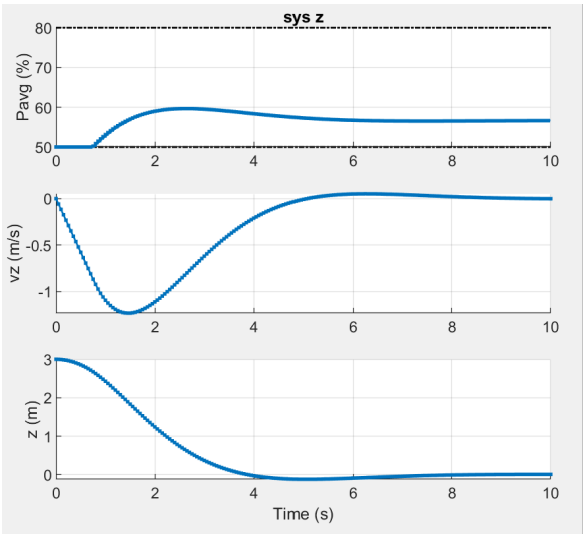


(d) closed-loop for y

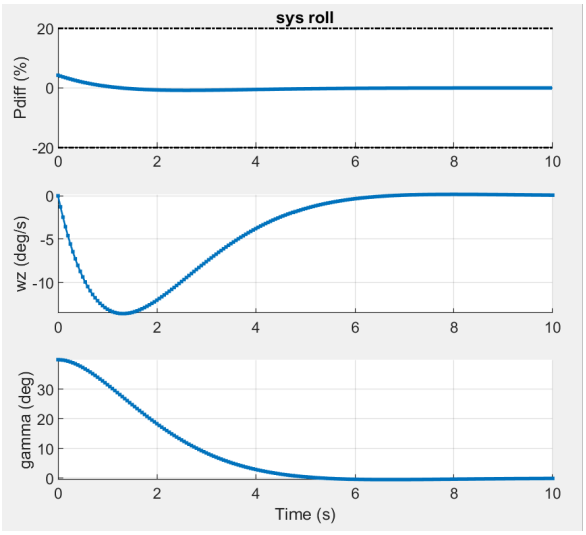
Figure 3.5: Open and closed-loop plot for x and y for 3.1



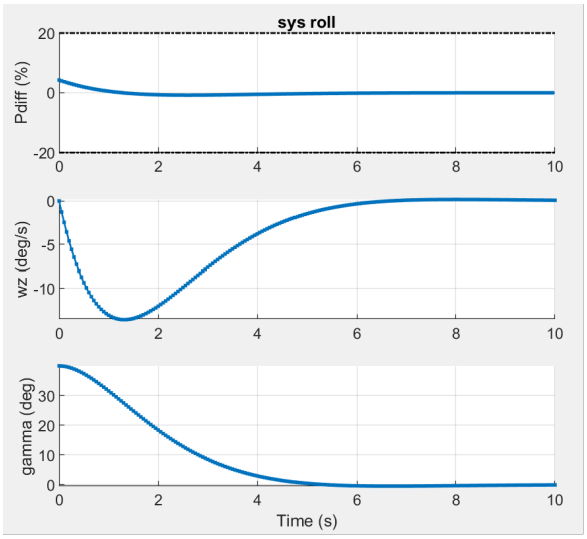
(a) open-loop for z



(b) closed-loop for z



(c) open-loop for roll



(d) closed-loop for roll

Figure 3.6: Open and closed-loop for z and roll for 3.1

3.2. TRACKER DESIGN PROCEDURE:

The tracker design procedure shares similarities with the controller design procedure outlined in Deliverable 3.1. Many steps, including the formulation of control objectives, constraint matrices, horizon, and cost matrices and the formulation of the optimization problem, have already been addressed in the previous section. However it is important to note that the LQR controller design and maximal invariant set computation have been removed as we do no longer need their presence in our tracker. Therefore, to avoid redundancy, these steps will not be repeated here. Instead, we focus on the specific aspects relevant to the tracker design:

1. **Reference Tracking Objectives and Steady-State Targets** In formulating objectives for precise trajectory tracking, we systematically defined essential state variables and corresponding reference values. An important step in achieving reference tracking involved computing steady-state targets (X_s and U_s). Formulas for X_s and U_s were derived from the rocket system's dynamics. The 'setup_steady_state_target' method in the `MpcControl` classes encapsulates this computation. Utilizing system dynamics matrices (`mpc.A` and `mpc.B`), the method incorporates constraints to ensure the steady-state aligns with the reference position, minimizing the squared norm of the steady-state input.

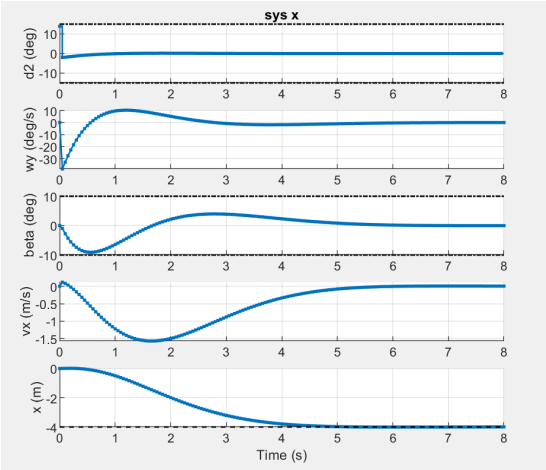
2. **Incorporation of Tracking Objectives into the Optimization Problem** Adjustments to the optimization problem were made to integrate the formulated reference tracking objectives. This process involved a modification in the objective function to ensure the rocket dynamically tracks the desired trajectory. Instead of the traditional quadratic cost term $U^T RU$, a novel objective function was implemented for the tracker system:

$$\text{obj} = (X(:,i) - x_{\text{ref}})^T \cdot Q \cdot (X(:,i) - x_{\text{ref}}) + (U(:,i) - u_{\text{ref}})^T \cdot R \cdot (U(:,i) - u_{\text{ref}})$$

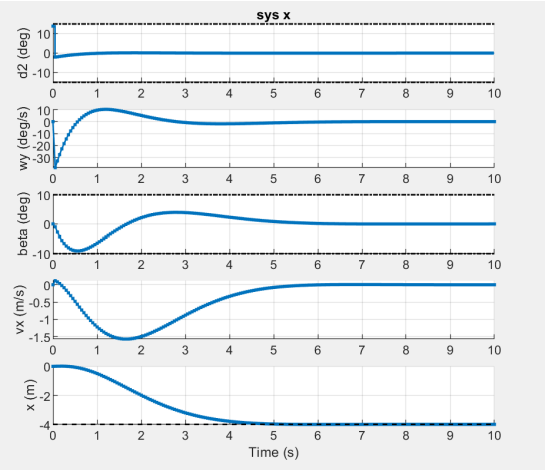
This new formulation aims to minimize the difference between the predicted state ($X(:,i)$) and the reference state (x_{ref}), as well as the difference between the predicted input ($U(:,i)$) and the reference input (u_{ref}).

3.3. TRACKER PARAMETER TUNING:

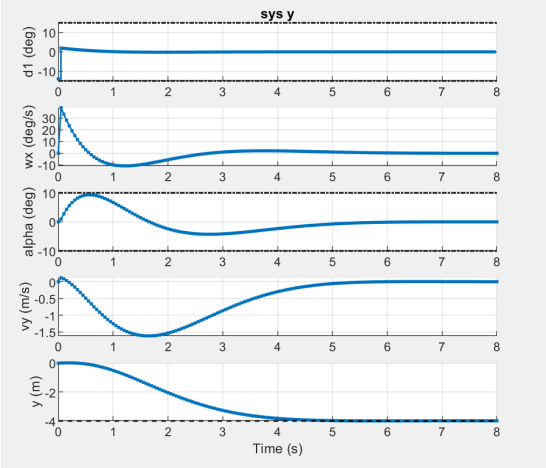
The tuning of parameters for the tracker, including Q, R, and H followed a similar rationale to those employed in the controller design. This involved tuning the values of Q and R matrices to achieve optimal trajectory following while respecting system pre-defined constraints.



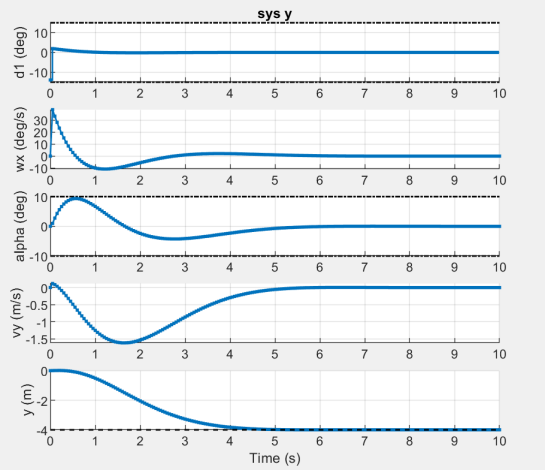
(a) open-loop for x



(b) closed-loop for x

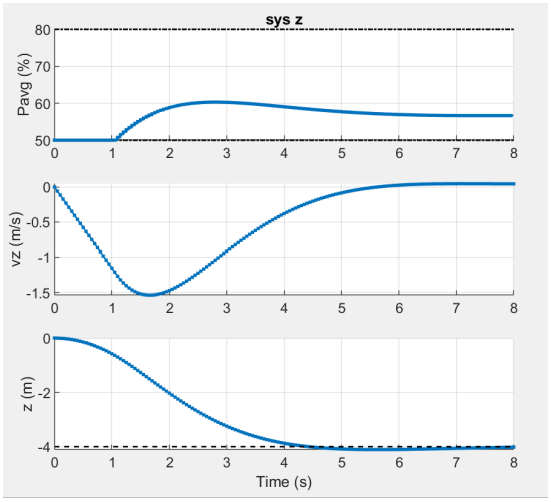


(c) open-loop for y

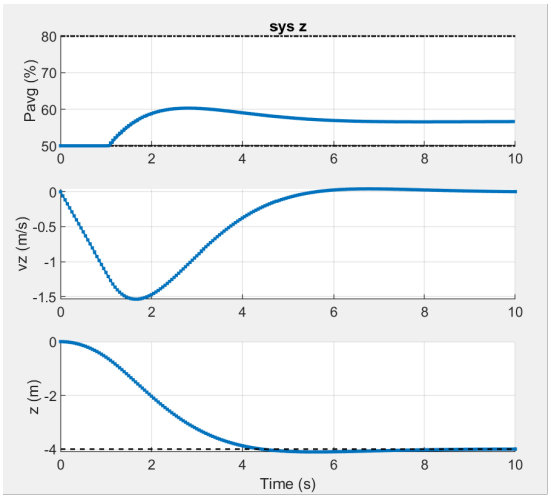


(d) closed-loop for y

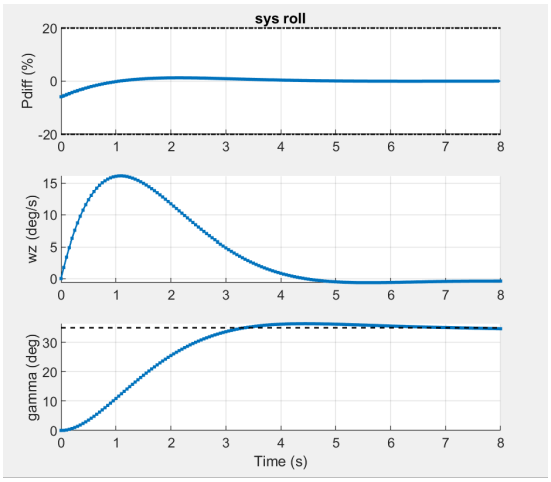
Figure 3.7: Open and closed-loop plot for x and y for 3.2



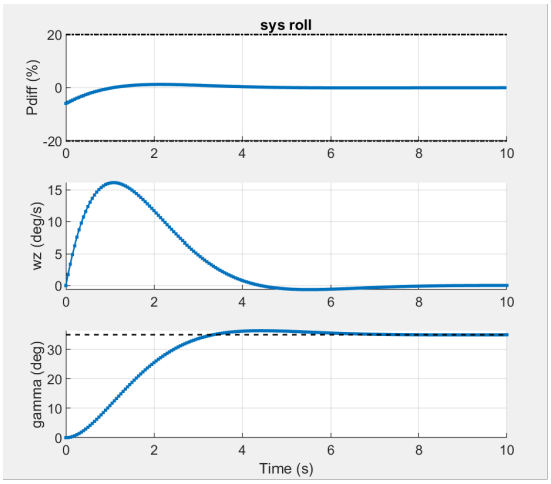
(a) open-loop for z



(b) closed-loop for z



(c) open-loop for roll



(d) closed-loop for roll

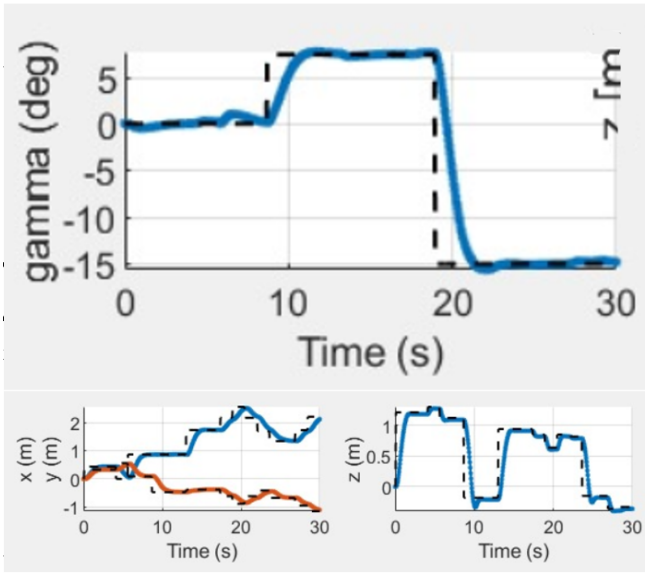
Figure 3.8: Open and closed-loop for z and roll for 3.2

4. DELIVERABLES 4.1 AND 4.2

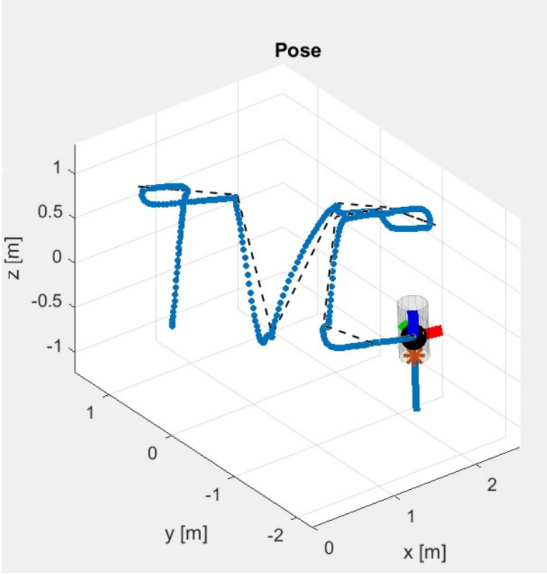
4.1. SIMULATION WITH NONLINEAR ROCKET

In this section, we use the four controllers for the nonlinear rocket to follow a given path. Since the trajectory is significantly more complex than the one in section 3.2 (Fig 4.1), the weight of our system have to be adapt. The first step was to increase the weights for the x, y, and z dimensions in order to enhance the system's ability to accurately follow the trajectory. To achieve improved tracking performance, these weights are significantly larger. Since the other weights are not so large, the rocket will attain a high speed to reach the reference path as quickly as possible. Achieving this speed requires the rocket to tilt. Considering that the constraints on the angles α and β have a small maximum angle (to maintain the approximation of the linearization), it is useful to introduce soft constraints to allow for temporary exceedance of the limit constraints. We decided to assign a high cost to the slack variable ("S" in the code) to minimize the size of violations. This increased cost also results in a longer duration before stabilization which does not disturb the system.

As we can see in the figure 4.1 (a), our controller adeptly follows straight lines but faces challenges when executing sharp turns, requiring more time to accomplish them. Indeed, it is impossible for the physical system to transition directly from 0 to 1. We observe that the linear system has its limitations; the angle cannot be taken too aggressively, as otherwise the constraints are exceeded by a considerable margin, even with the use of soft constraints. In conclusion, using a linear system to follow a given



(a) Tracking performance of the controller



(b) Position of the rocket

Figure 4.1

trajectory with a nonlinear rocket is feasible but comes with limitations.

5. DELIVERABLES 5.1 AND 5.2

5.1. OFFSET-FREE TRACKING CONTROLLER

Tracker Design Procedure

Several essential changes were made to the controller architecture to address the offset in the z-dimension caused by the change in the rocket's mass causing. To correct this offset we needed to implement an observer to our system by computing the augmented matrices $\overline{A}$, $\overline{B}$, and $\overline{C}$ as follows :

$$\overline{A} = \begin{bmatrix} A & B \\ 0 & 1 \end{bmatrix}$$

$$\overline{B} = \begin{bmatrix} B \\ 0 \end{bmatrix}$$

$$\overline{C} = \begin{bmatrix} C & 1 \end{bmatrix}$$

We then proceeded to create the estimator matrix (the L matrix) using the **place** function in matlab as shown below :

$$L = -\text{place}(A'_{\text{bar}}, C'_{\text{bar}}, [0.1, 0.3, 0.5])'$$

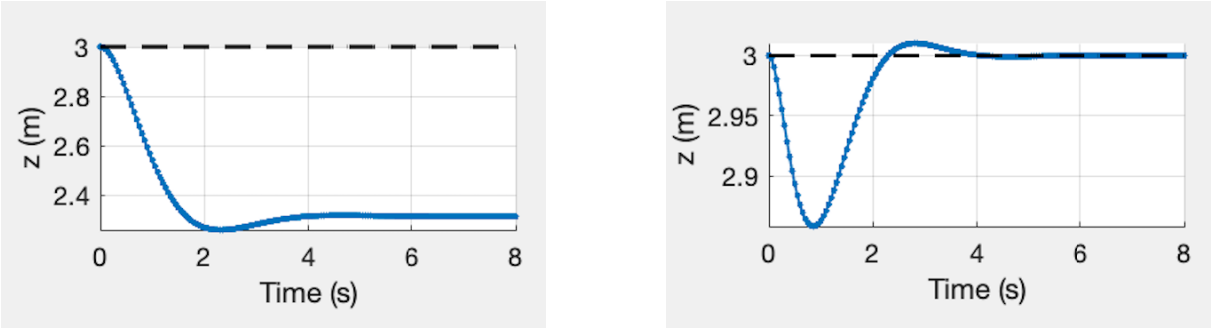
where the poles presented here ([0.1, 0.3, 0.5]) were hand picked and tuned through a trial-and-error approach as to create an observable with fast dynamics to compensate for the induced disturbance all while maintaining the stability of the system.

Offset-Free Parameter Tuning

We needed to adapt the controller to handle the offset-free tracking requirement in the z-dimension.Which meant that the tuning parameters for the cost matrices Q and R were re-evaluated and adjusted to accommodate the new system with the different mass introduced by the disturbance estimate. We used as in other parts of our project the trial-and-error technique and a lot of testing to be able to come up with the tuning matrices presented below showing our final parameter choice for the offset-free tracking. These parameters are responsible for to deliver a smooth tracking and offset rejection without a significat overshoot all while reducing the convergance time.

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 500 \end{bmatrix}$$

$$R = 1$$



(a) Influence of the mass change on the controller

(b) Offset-free tracking

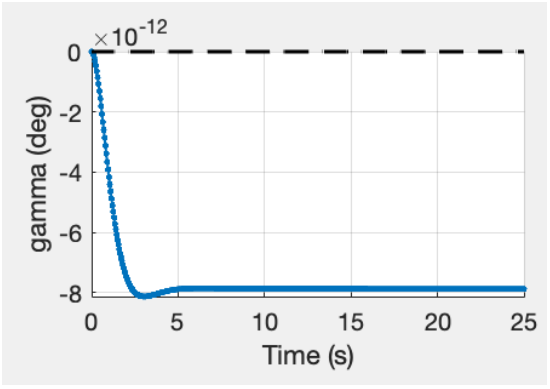
Figure 5.1: In the figures shown above we present the difference between the left without the estimator/compensator where we can see that we have a constant offset that we can't reject of about 0.7 in the z-axis however in the right figure we can clearly see that the perturbance cause is quickly rejected and the system can follow the reference perfectly withing 5 seconds

5.2. SYSTEM WITH CHANGING MASS

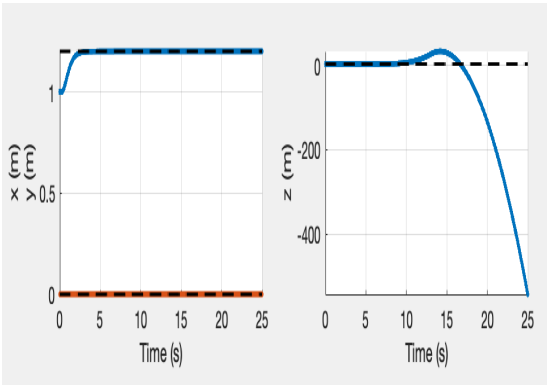
In the initial seconds of the simulation, a tracking offset in height may persist despite the presence of the estimator. This phenomenon is caused due to the difficulties associated with estimating the changing mass of the rocket. The mass variations introduce uncertainties in the dynamics, leading to discrepancies between the estimated and actual states and that being that the changing mass is not taken into account when computing the observer with the initial model matrices mpc.A, mpc.B and mpc.C. That's

why using only he initial state will not help the observer account for the change in mass in it's prediction and perturbation rejection thus the offset seen in the plots shown below.

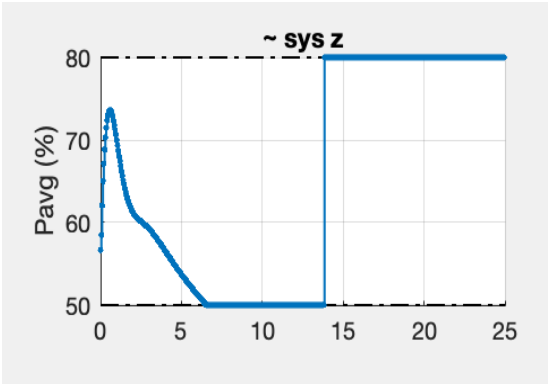
To achieve offset-free tracking, we need to handle the changing mass case. One approach can be to enhance our observer's model to take into account a more accurate representation of the mass dynamics when building the augmented matrices and the observer matrix. This can involve refining the mass estimation process by incorporating a more sophisticated model that captures the mass variations more effectively at the cost of a more complex implementation.



(a) The thrust-dependent tracking γ system



(b) The thrust-dependent tracking X/Y/Z systems



(c) The average thrust power input to the rocket

Figure 5.2: The figures shown above describe the state of the system simulated for 25 seconds with a thrust dependant model where rocket's mass_rate was equal to -0.27 and the initial mass was 2.13 along with the average thrust power given to the rocket

Due to the timevarying nature of our system we can observe multiple phenomenas during our simulation such as

1. The fact that the varying mass creating a larger perturbance than the one for which we have computed it for, our rocket can follow the reference but it does so with an offset always present not as in the case of the offset-free tracking, we can no

longer reject the offset correctly.

2. Due to the decrease of the mass we can see that even thought the thrust is maintained for quite some time at 50 the rocket can still hover at a constant altitude without needing the initial 56.6667 as with the full mass.

3. Then once we loose all fuel of the rocket we can noticed a huge increase in the thrust given to the rocket along with a huge jump in the z direction showing the tremendous altitude gain around 15 seconds

4. Finally as expected after the increase in the rocket thrust due to the decreased mass, the rocket have no more fuel to burn and so it gets into a free fall state as shown between 15 and 25 seconds between which the input needed to compensate was the maximum at 80 as shown in the figure however this was not possible due to the fact that the rocket had no more fuel.

6. DELIVERABLES 6.1 AND 6.2

6.1. NONLINEAR MPC

In the following sections we will be looking into the creation of our NMPC controller for the rocket system. The first important milestone was the definition of our cost which is a quadratic funtoin same as the linear MPC controler represented as follows:

$$cost = (X_{\text{sym}} - ref_{\text{sym}})' \cdot Q \cdot (X_{\text{sym}} - ref_{\text{sym}}) + (U_{\text{sym}} - us)' \cdot R \cdot (U_{\text{sym}} - us) \quad (6.1)$$

Then we needed to compute our equality and inequality constraints and to do so we needed to discretize our system using the Runge Kutta 4 (see algorithm 6.3) implemented directly in the main loop, then using the result of such algorithm to predict the next step thus giving us our equality constraints as :

$$eq_{\text{constr}} = [eq_{\text{constr}}; X_{\text{sym}}(:, k+1) - X_{\text{next}}] \quad (6.2)$$

where X_{next} is the result of the discretization of our dynamic model by the Runge Kutta 4 algorithm.

Once we compute the cost and equality constraints for the system we move to the computation of our terminal cost and inequality constraints. The way we did it is by linearization and discretizing the model of the system using the functions *rocket.linearize(xs, us)* and *c2d()*, then once we have our desired matrices we computed the gain **K** and the final weight **Qf**, we compute the final invariant set **Xf** by iteration using the same technique as explained in **??**. Once found we then finalise our

terminal cost and our inequality constraints as shown below :

$$cost = (X_{\text{sym}} - ref_{\text{sym}})' \cdot Qf \cdot (X_{\text{sym}} - ref_{\text{sym}}) \tag{6.3}$$

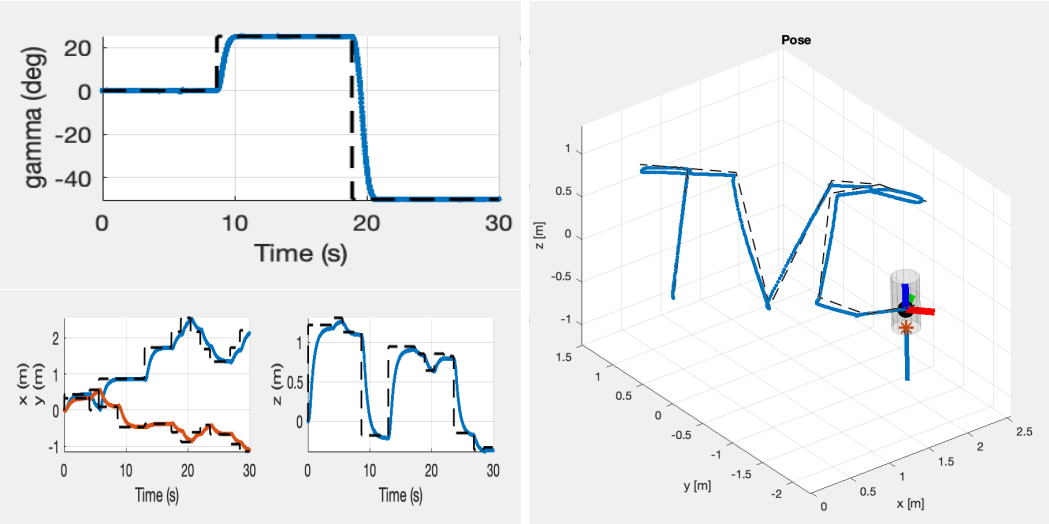
$$ineq_{\text{constr}} = [ineq_{\text{constr}}; Ff * X_{\text{sym}}(:, N) - ff] \tag{6.4}$$

To attain the optimal solution we tuned the matrices Q and R in an iterative procesure while keeping in mind the importance of each of them, for the Q we decided to give more weight to the variables that mean the most which are the **x**,**y** and **z** variables which are required to follow a certain reference. After them we tried to limit the angular velocities to reject any aggressive manoeuvres. The R however was mainly used to keep the inputs within their respective bounds as we know by now that the higher the R the less aggressive the inputs will be but the longer it will take to stabilise the system.

6.1.1. Nonlinear Controller vs Linear Controllers

	Nonlinear	**Linear**
Pros	- Capable of handling the full nonlinear dynamics of our complex model. - Suitable for a wider range of operating conditions with fewer limiting factors (e.g., $\lvert\alpha \,\&\, \beta\rvert < 10°$).	- Computationally more intensive due to the fact that we no longer divide into sub-systems. - Design and tuning complexity caused by the increase in the size of the Q and R matrices and the appearance of complex interactions between variables such as ω affecting α.
Cons	- Computation superiority due to small distributed computations that are less demanding. - Simplicity in design and implementation and ease of visualization and plotting (maximum dimension complexity is 4 dimensions).	- Limited to regions where linearization is accurate because otherwise the model corresponds no more to the reality. - May not perform well under different and extreme conditions due to assumptions and simplifications.

Table 6.1: Comparison between Nonlinear and Linear Controllers.



(a) References **(b)** TVC

Figure 6.1: These plot illustrates the performance of the NMPC controller under the tracking of the reference of the TVC. In the lower left plot : the orange curve correponds to the y while the blue corresponds to the x. IN the upper left plot we can see the tracking for the γ, and in the small lower right plot we have the tracking of the z. Finally the right big plot is the tracking of the whole TVC path.

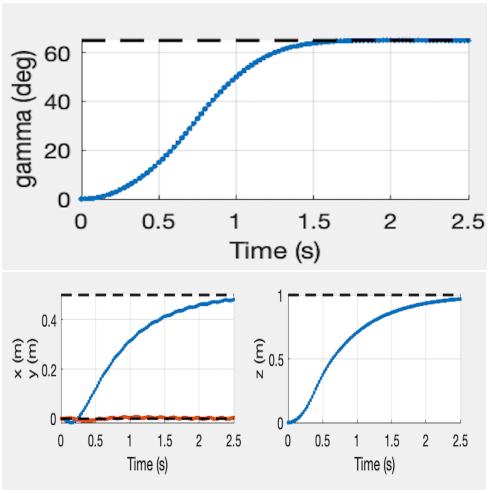
6.2. NMPC WITH DELAY COMPENSATION

Delay is a dangerous problem that can easily destabilize our system. In an ideal world once we estimate the next position and compute our optimal solution we immediately apply the optimal input however in real life, computing the optimal solution can take some time thus leading to errors. After several trials we have studied the evolution of our NMPC with a fixed horizon of 4s and varied the delay from **0 steps** (in the ideal case) to **3 steps** where we see divergence and loss of stability of our system.

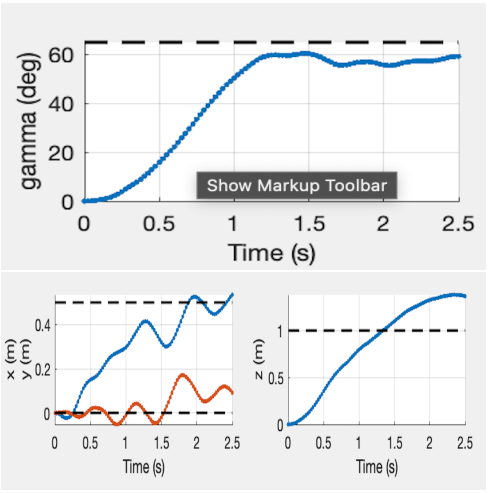
In the figure shown below we noted that after a delay of **1 step** our NMPC could keep tracking the reference with a very oscillating behavior for the beta inputs, from a delay of **2 steps** we can already see that the system's performance is really bad, the controller tries to follow the reference given but without success and finally once we reached **3 steps** delay we found that we lost stability and the system can no longer be controlled. Each delay step corresponds to **25 millisecond** which can quickly accumulate becoming crucial for the rocket knowing that the NMPC optimal solution computation takes **hundreds of milliseconds**.

6.2.1. No/Partial/Full delay compensation

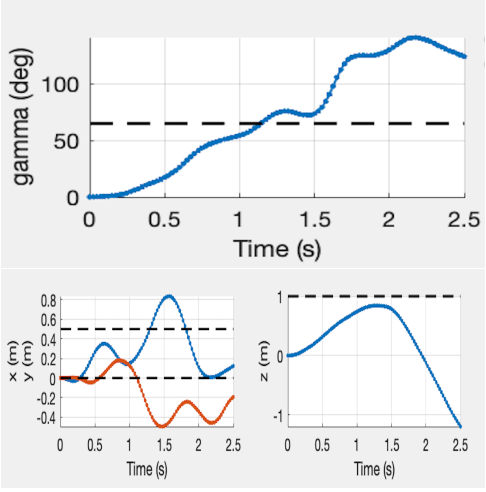
In the figures below we will be showing the difference the delay compensator can have on the stability and performance of our system. Having no compensator at all can cause huge problems ranging from decreasing performance to system instability. Having a partial delay compensator can improve our performance however it is not optimal in the sense where it can be met by vibrations and overshoot. The final and optimal case we will be showing is the full compensator where we can track the reference even with varying delay.



(a) 1 step delay without compensator

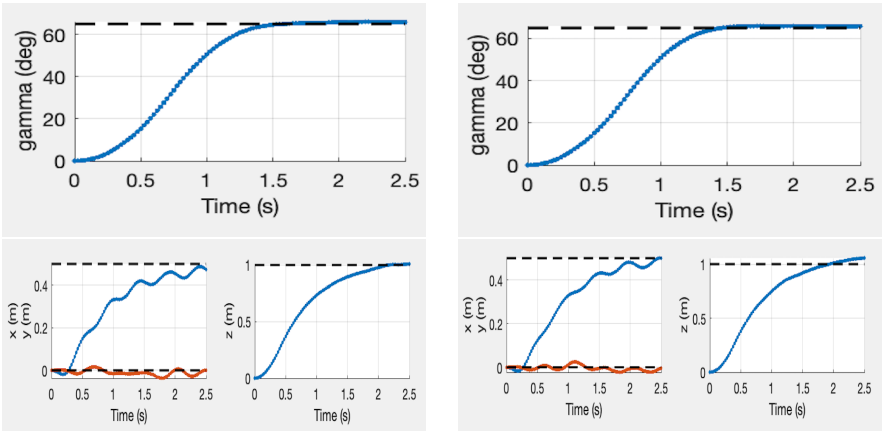


(b) 2 steps delay without compensator



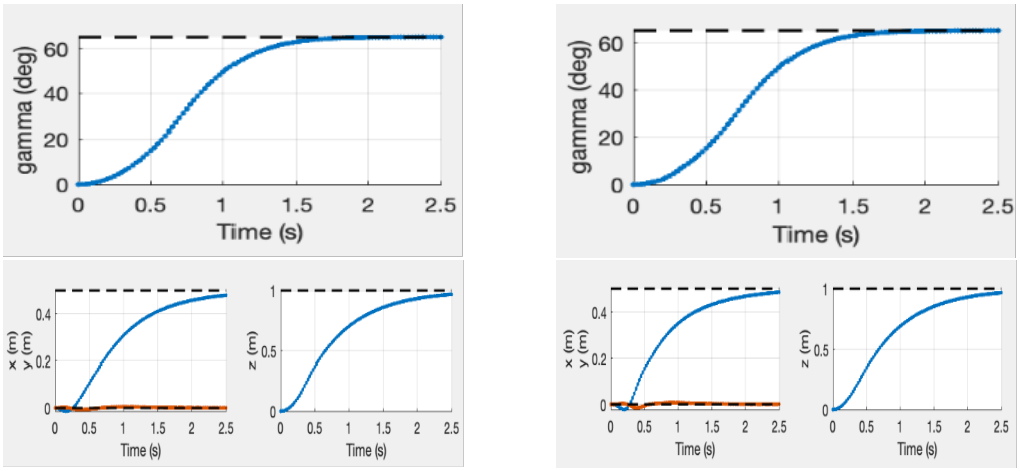
(c) 3 steps delay without compensator

Figure 6.2: The above plots show the effect of the different delays on our control system loosing performance after 2 delay steps (equivalent to 50ms) and loosing convergence after 3 delay steps (equivalent to 75ms)

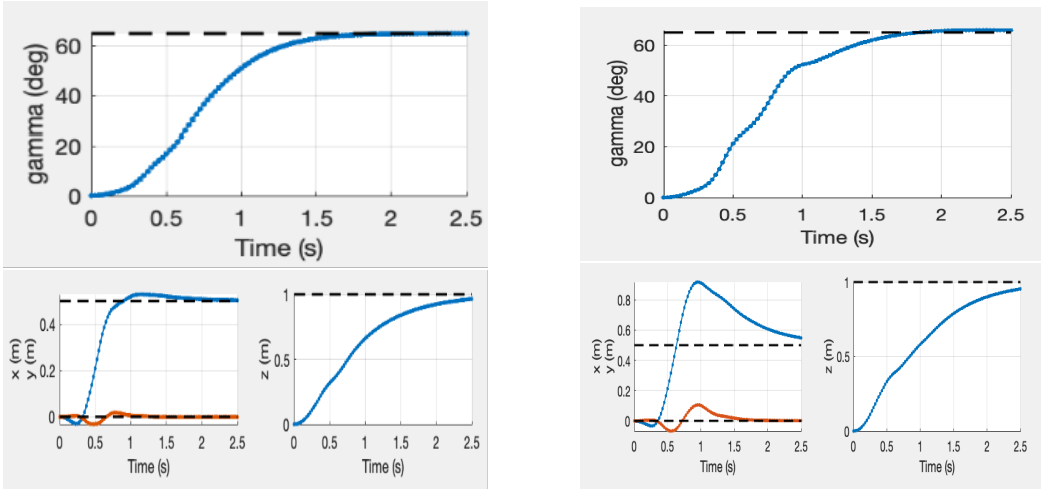


(a) actual delay=4, compensated delay=3 **(b)** actual delay=3, compensated delay=2

Figure 6.3: We can see an improvement in the performance of our system with the delays induces by 3 delay steps and the 4 delay steps however we also notices that the system is still not at it's optimal state as we have some vibrations in the x and y states along with an overshoot of the z state in the (b) lower right plot. This showcases the problems with compensating for a static delay without taking into account the real delays of the system.



(a) 3 step delay compensation **(b)** 5 steps delay compensation



(c) 7 steps delay compensation **(d)** 9 steps delay compensation

Figure 6.4: Finally we have deployed the fully delay-compensating controller which resulted in a smooth following for our reference even for delays that are up to 9 steps which corresponds to a delay of 225 milliseconds

APPENDIX

6.3. RUNGE KUTTA 4 ALGORITM

Given the initial value problem:

$$\frac{dx}{dt} = rocket.f(), \quad X(t_0) = x_0$$

The Runge-Kutta 4th order method is expressed as follows:

$$\begin{aligned}
k1 &= rocket.f(X_{\text{sym},k}, U_{\text{sym},k}) \\
k2 &= rocket.f(X_{\text{sym},k} + \frac{h}{2} * k1, U_{\text{sym},k}) \\
k3 &= rocket.f(X_{\text{sym},k} + \frac{h}{2} * k2, U_{\text{sym},k}) \\
k4 &= rocket.f(X_{\text{sym},k} + h * k3, U_{\text{sym},k}) \\
X_{\text{next}} &= X_{\text{sym},k} + \frac{h}{6}(k1 + 2 * k2 + 2 * k3 + k4)
\end{aligned}$$

Where:

- $X_{\text{sym},k}$ is the current value of the solution,
- h is the time step,
- rocket.f is the system continous dynamics.